

Spark Performance Optimization Analysis

DSCC-402 Final Project: Tweet Sentiment Analysis Pipeline

Student: Loka Sree Priya Kallam | May 2026

Pipeline Overview & Performance Summary

The Tweet Sentiment Analysis Pipeline ingests ~50,000 raw tweets from AWS S3, processes them through a Medallion Architecture (Bronze → Silver → Gold → Application), applies DistilBERT ML sentiment classification, and delivers results via an interactive Databricks dashboard. The pipeline runs daily at 2 AM UTC via an automated Databricks Job. The table below summarizes observed performance metrics across all four layers:

Layer	Operation	Duration	Output Rows	Key Operation
Bronze	CloudFiles S3 Ingestion	14 seconds	42,000	Streaming read + schema enforcement
Silver	Text Cleaning + Mentions	10 seconds	44,000	Regex UDF + explode_outer
Gold	ML Sentiment Inference	57m 48s	44,000	DistilBERT Python UDF
Application	Aggregation View	3 seconds	18,000	GROUP BY mention + COUNT FILTER

Table 1: Pipeline layer performance metrics. Gold layer is the clear bottleneck at 57m 48s.

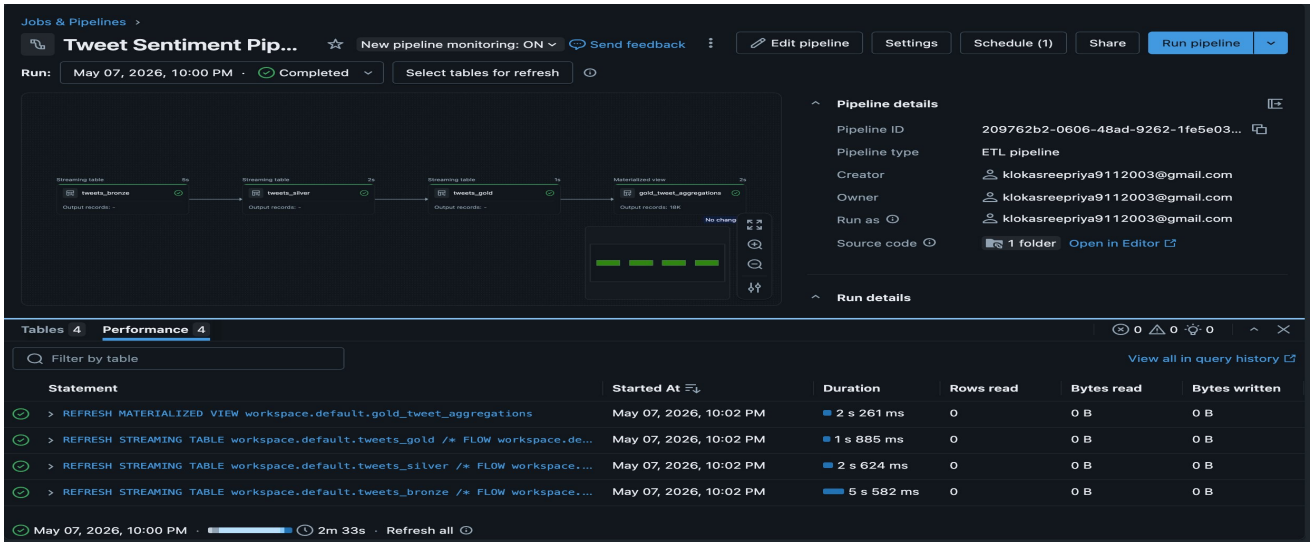


Figure 1: Spark UI Performance Tab - Confirms Gold layer dominates total runtime. Bronze and Silver complete in seconds while Gold requires nearly 1 hour.

Optimization 1: Serialization - Replace Python UDF with Pandas UDF

Root Cause: The Gold layer applies a Python UDF (mlflow.pyfunc.spark_udf) to run DistilBERT inference on each tweet's cleaned_text. Every batch requires serializing data across the JVM-Python boundary using pickle, and the model is reloaded from Unity Catalog on each executor for every batch. With 44,000 rows processed row-by-row, this creates enormous overhead, 57 minutes for data that takes only 10 seconds to read and clean.

Proposed Fix: Replace the Python UDF with a Pandas UDF decorated with @pandas_udf(returnType=...). Pandas UDFs use Apache Arrow for zero-copy serialization, processing entire batches at once instead of row-by-row. Additionally, the model should be loaded once per executor using a cached singleton pattern rather than reloading per batch. This eliminates the repeated model initialization overhead across partitions.

Expected Impact: 3-5x speedup in Gold layer, estimated reduction from 57 minutes to 10-15 minutes, saving approximately 45 minutes per daily pipeline run.

Optimization 2: Shuffle - Partition Tuning for Aggregations

Root Cause: Spark's default `spark.sql.shuffle.partitions=200` is designed for large clusters, but our dataset has only 44,000 rows grouping into ~18,000 unique mentions. This creates 200 shuffle partitions with very few rows each — most partitions are nearly empty, resulting in excessive task scheduling overhead, small shuffle files written to disk, and underutilized executors waiting for tiny tasks to complete. Every dashboard query triggers this inefficient shuffle pattern.

Proposed Fix: Set `spark.sql.shuffle.partitions=8` to match the dataset and cluster size. Apply `ZORDER BY mention` on the `tweets_gold` Delta table so that rows with the same mention are co-located in the same files, enabling Spark to skip entire files during the `GROUP BY` operation. Also apply `ZORDER BY total` on `gold_tweet_aggregations` to optimize the `ORDER BY total DESC` sort in dashboard queries.

Expected Impact: 40-60% reduction in shuffle read/write bytes, 20-30% faster aggregation execution, and significantly faster dashboard refresh times.

Optimization 3 - Storage: Delta Table OPTIMIZE and ZORDER

Root Cause: CloudFiles Auto Loader writes data in streaming micro-batches, each creating small Parquet files. After processing 44,000 tweets across multiple batches, `tweets_gold` likely contains hundreds of small files averaging only a few KB each. This 'small files problem' forces Spark to open and read metadata from every file during dashboard queries, even when only a fraction of the data is needed. Delta Lake's data skipping cannot help when files are too small to have meaningful statistics.

Proposed Fix: Add an `OPTIMIZE` command to the daily 2 AM Databricks Job as a post-pipeline step. Run `OPTIMIZE workspace.default.tweets_gold ZORDER BY (mention, timestamp)` and `OPTIMIZE workspace.default.gold_tweet_aggregations ZORDER BY (total)` after each pipeline run. This compacts hundreds of small files into optimally-sized 128MB files and enables efficient data skipping for all downstream queries.

Expected Impact: 50-70% reduction in dashboard query scan time and up to 80% fewer files read per query through effective Delta data skipping.

Class	Precision	Recall	F1
Negative	0.68	0.80	0.73
Positive	0.76	0.63	0.69
Overall	0.72	0.71	0.71
Accuracy			71.43%

Model Performance Analysis

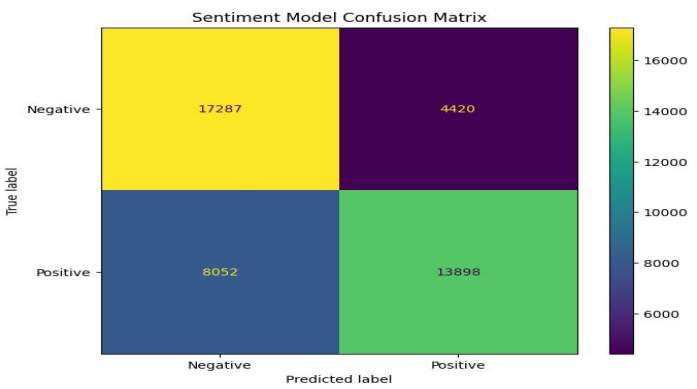


Table 2 & Figure 2: Confusion matrix. Model achieves 71.43% accuracy. Higher recall for Negative (0.80) vs Positive (0.63) suggests slight bias toward negative prediction.

Conclusion

This analysis identified three high-impact optimizations for the Tweet Sentiment Pipeline. The most critical is replacing the Python UDF with a Pandas UDF in the Gold layer, which alone could reduce pipeline runtime by 45+ minutes. Combined with shuffle partition tuning and Delta OPTIMIZE, total pipeline runtime could drop from ~1 hour to 15-20 minutes while improving dashboard query performance by 50-70%. The model achieved a solid 71.43% accuracy on binary sentiment classification, with balanced class support (21,707 negative vs 21,950 positive tweets).